🏗️ Backend Architecture - Complete Packages Guide

Master All Backend Packages

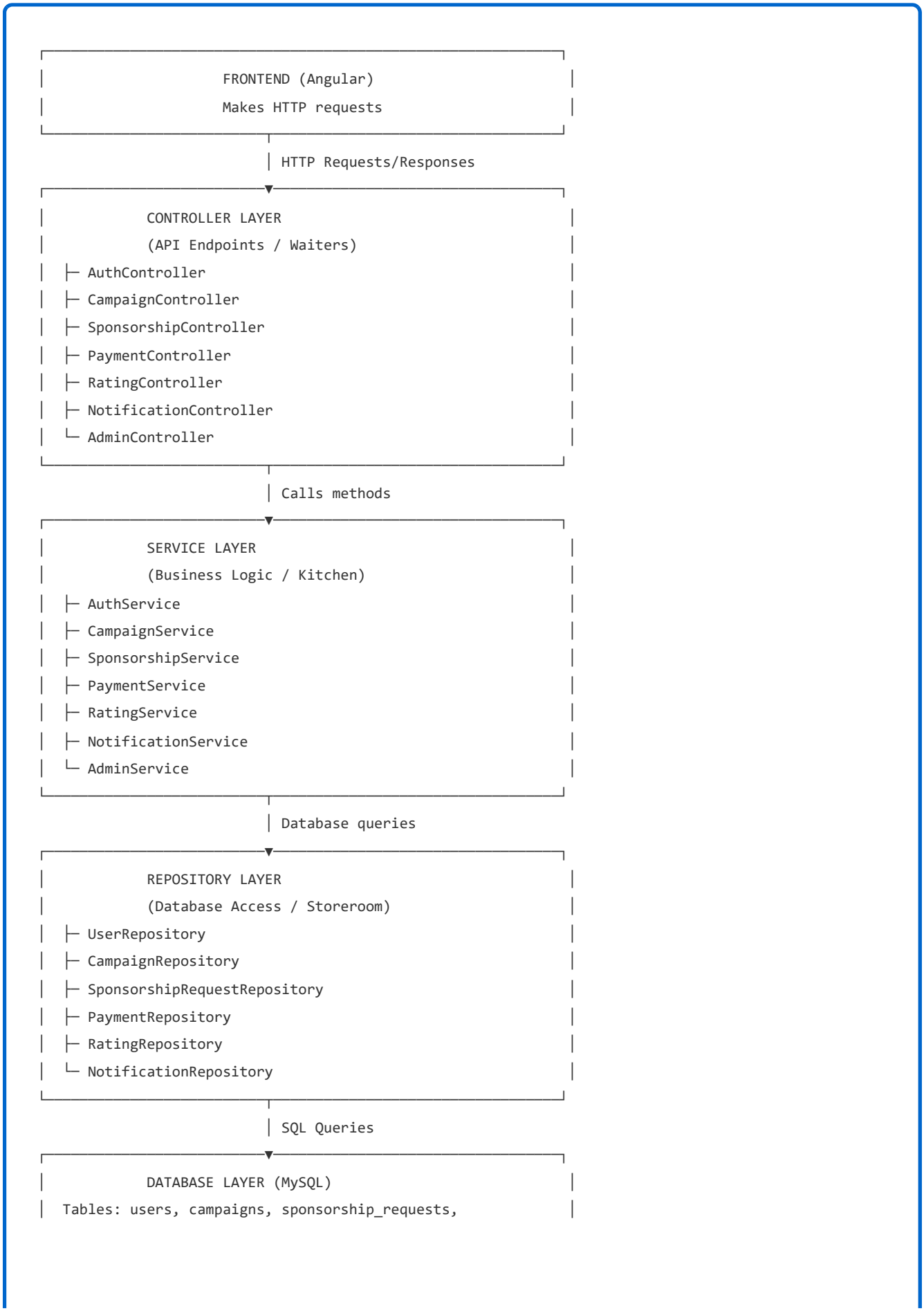
This comprehensive guide explains every package in your backend: what they do, their responsibilities, the classes they contain, how they connect, and what role each plays in the application architecture.

📑 Complete Table of Contents

1️⃣ Backend Architecture Overview

The Three-Tier Architecture

Your backend uses the classic **three-tier architecture** pattern. Think of it like a restaurant:



```
│          payments, notifications, ratings                    │
└──────────────────────────────────────────────────────────────┘
```

Supporting Packages

- **DTO Package:** Formats data for API requests/responses
- **Entity Package:** Represents database tables as Java objects
- **Security Package:** Handles JWT authentication
- **Exception Package:** Manages errors globally
- **Config Package:** Spring Boot configuration (covered earlier)

2️⃣ Entity Layer - Database Models

What is the Entity Package?

Entities are **Java classes that map to database tables**. Each entity = one table. Each field = one column.

> **Path:** src/main/java/com/myapp/sponsorshipapp/entity/
>
> **Contains:** Entity classes + Enum classes
>
> **Purpose:** Represent domain objects and database schema

Entity Classes in Your Project

Entity Class	Database Table	Purpose	Key Fields
User.java	users	Store user accounts	id, name, email, password, role, bio, profileImage
Campaign.java	campaigns	Store promotional campaigns	id, name, description, budget, platform, dates, brand_id
SponsorshipRequest.java	sponsorship_requests	Track influencer applications	id, influencer_id, campaign_id, proposal, status
Payment.java	payments	Record transactions	id, campaign_id, influencer_id, brand_id, amount, status
Notification.java	notifications	Store alert messages	id, user_id, title, message, isRead

Rating.java	ratings	Store reviews & feedback	id, campaign_id, rater_id, rated_id, score, feedback

Enum Classes (not tables)

Enum	Values	Used In
Role.java	ADMIN, BRAND, INFLUENCER	User.role
CampaignStatus.java	ACTIVE, PAUSED, COMPLETED, CANCELLED, EXPIRED	Campaign.status
PaymentStatus.java	PENDING, COMPLETED, FAILED, REFUNDED	Payment.status
RequestStatus.java	PENDING, ACCEPTED, REJECTED, COMPLETED	SponsorshipRequest.status

3️⃣ DTO Package - Data Transfer Objects

What is a DTO?

DTO = Data Transfer Object. DTOs are **simple Java classes that hold data for API communication**. They transfer data between frontend and backend.

> **Path:** src/main/java/com/myapp/sponsorshipapp/dto/
>
> **Key Difference:**
> • **Entity:** Database representation (with relationships)
> • **DTO:** API communication (simplified, no relationships)
>
> **Why separate?** Keep API simple + protect sensitive data

DTO Classes - Complete List

DTO Class	Purpose	Used For	Key Fields
LoginRequest.java	User login data	POST /api/auth/login	email, password
RegisterRequest.java	User registration data	POST /api/auth/register	name, email, password, role
AuthResponse.java	Login response	Returns after login/register	token, id, name, email, role
CampaignRequest.java	Campaign creation/edit	POST/PUT /api/campaigns	name, description, platform, budget, dates

SponsorshipApplicationRequest.java	Influencer application	POST /api/influencer/apply	campaignId, proposal
WorkSubmissionRequest.java	Influencer work submission	POST /api/influencer/submit-work	sponsorshipId, workDescription
PaymentRequest.java	Payment processing	POST /api/brand/payments	sponsorshipId, amount
RatingRequest.java	Submit review/rating	POST /api/campaigns/rate	campaignId, ratedUserId, score, feedback
ChangePasswordRequest.java	Change password	POST /api/auth/change-password	oldPassword, newPassword, confirmPassword
ApiResponse.java	Generic API response wrapper	All endpoints	success, message, data
DashboardStats.java	Dashboard statistics	GET /api/admin/stats	totalCampaigns, totalUsers, totalEarnings, etc

Example: LoginRequest DTO

LoginRequest.java - What It Does

```
@Data public class LoginRequest { @Email(message = "Invalid email format") @NotBlank(message = "Email is required") private String email; @NotBlank(message = "Password is required") private String password; } // When user logs in: // Frontend sends JSON: {"email":"john@gmail.com","password":"pass123"} // Spring automatically converts JSON to LoginRequest object // @Email and @NotBlank validate the data // If valid, AuthService processes it
```

Example: AuthResponse DTO

AuthResponse.java - What It Returns

```
@Data @AllArgsConstructor public class AuthResponse { private String token; // JWT token private String type = "Bearer"; private Long id; // User ID private String name; // User name private String email; // User email private String role; // User role } // After successful login, AuthService returns AuthResponse // Frontend receives JSON: { // "token": "eyJhbGciOiJ...", // "type": "Bearer", // "id": 1, // "name": "John", // "email": "john@gmail.com", // "role": "INFLUENCER" // }
```

ApiResponse - Universal Response Wrapper

ApiResponse.java - Consistent Responses

```
@Data @AllArgsConstructor public class ApiResponse { private boolean success; // true or false private String message; // "Campaign created successfully" private Object data; // Actual data (campaign object, list, etc) } // Usage Examples: // Success: {"success":true,"message":"Campaign created","data":{...campaign object...}} // Error: {"success":false,"message":"Campaign not found","data":null} // This ensures CONSISTENT response format across all endpoints // Frontend knows exactly what to expect!
```

4️⃣ Repository Layer - Database Access

What is a Repository?

Repository = Data Access Object (DAO). It handles all database queries. Instead of writing SQL, you write simple Java methods that Spring Data JPA converts to SQL automatically.

> **Path:** src/main/java/com/myapp/sponsorshipapp/repository/
>
> **Technology:** Spring Data JPA + Hibernate
>
> **Benefit:** No SQL writing needed! Just method names!

Repository Classes

Repository	Manages	Key Methods
UserRepository	User entity	findByEmail(), existsByEmail(), findByRole()
CampaignRepository	Campaign entity	findByBrand(), findByStatus(), findAll()
SponsorshipRequestRepository	SponsorshipRequest entity	findByInfluencerAndCampaign(), findByStatus()
PaymentRepository	Payment entity	findByStatus(), findByCampaign(), findByInfluencer()
NotificationRepository	Notification entity	findByUser(), findByUserAndIsReadFalse()
RatingRepository	Rating entity	findByCampaign(), findByRaterId(), findByRatedId()

Example: UserRepository

UserRepository.java - Database Queries

```
@Repository public interface UserRepository extends JpaRepository<User, Long> { // Spring Data JPA understands these method names and creates SQL! Optional<User> findByEmail(String email); // SQL: SELECT * FROM users WHERE email = ? // Returns Optional (might be empty) boolean existsByEmail(String email); // SQL: SELECT COUNT(*) FROM users WHERE email = ? // Returns true/false boolean existsByNameIgnoreCase(String name); // SQL: SELECT COUNT(*) FROM users WHERE LOWER(name) = LOWER(?) List<User> findByRole(Role role); // SQL: SELECT * FROM users WHERE role = ? // Returns list of all users with this role } // Benefits: // ✓ No SQL to write // ✓ Type-safe (return types are Java objects) // ✓ Automatic parameter binding (prevents SQL injection) // ✓ Built-in pagination and sorting
```

5 Service Layer - Business Logic

What is a Service?

Service = Business Logic Layer. Services contain all the rules and decisions. They use repositories to access data and apply business logic on top.

> **Path:** src/main/java/com/myapp/sponsorshipapp/service/
>
> **Contains:** Business logic for each domain
>
> **Principle:** Fat service layer, thin controllers

Service Classes Overview

Service Class	Responsibility	Key Methods (Approx)
AuthService	Authentication & registration	register(), login(), changePassword(), getCurrentUser()
CampaignService	Campaign management	createCampaign(), updateCampaign(), getCampaignById(), getActiveCampaigns()
SponsorshipService	Application management	applyToCampaign(), approveApplication(), submitWork(), completeWork()
PaymentService	Payment processing	processPayment(), getPaymentStatus(), refundPayment()
NotificationService	Notification management	sendNotification(), getNotifications(), markAsRead()
RatingService	Review & ratings	submitRating(), getRatings(), getAverageRating()
AdminService	Admin operations	getDashboardStats(), getAllUsers(), suspendUser()

Example: AuthService - Detailed Breakdown

AuthService.java - Key Methods

```
@Service public class AuthService { @Autowired private UserRepository
userRepository; @Autowired private PasswordEncoder passwordEncoder; //
BCrypt @Autowired private AuthenticationManager authenticationManager;
@Autowired private JwtTokenProvider tokenProvider; // ===== REGISTER
METHOD ===== public AuthResponse register(RegisterRequest request) { //
1. Normalize email & name String normalizedEmail =
request.getEmail().toLowerCase().trim(); // 2. Check if email already
exists if (userRepository.existsByEmail(normalizedEmail)) { throw new
RuntimeException("Email already registered"); } // 3. Create new User
object User user = new User(); user.setName(request.getName());
user.setEmail(normalizedEmail); // 4. Hash password with BCrypt
user.setPassword(passwordEncoder.encode(request.getPassword())); // 5.
Set role (ADMIN, BRAND, or INFLUENCER)
user.setRole(Role.valueOf(request.getRole().toUpperCase())); // 6. Save
to database userRepository.save(user); // 7. Authenticate the user
Authentication authentication = authenticationManager.authenticate( new
UsernamePasswordAuthenticationToken(normalizedEmail,
request.getPassword()) ); // 8. Generate JWT token String token =
tokenProvider.generateToken(authentication); // 9. Return response with
token return new AuthResponse(token, user.getId(), user.getName(),
user.getEmail(), user.getRole().name()); } // ===== LOGIN METHOD =====
public AuthResponse login(LoginRequest request) { String email =
request.getEmail().toLowerCase().trim(); // 1. Authenticate with Spring
Security // AuthenticationManager uses passwordEncoder to compare
passwords Authentication authentication =
authenticationManager.authenticate( new
UsernamePasswordAuthenticationToken(email, request.getPassword()) ); //
2. Set authentication in SecurityContext
SecurityContextHolder.getContext().setAuthentication(authentication); //
3. Generate JWT token String token =
tokenProvider.generateToken(authentication); // 4. Find user from
database User user = userRepository.findByEmail(email) .orElseThrow(() ->
new RuntimeException("User not found")); // 5. Return response return new
AuthResponse(token, user.getId(), user.getName(), user.getEmail(),
user.getRole().name()); } // ===== GET CURRENT USER ===== public User
getCurrentUser() { // Get authenticated user from SecurityContext
Authentication auth =
SecurityContextHolder.getContext().getAuthentication(); String email =
auth.getName(); return userRepository.findByEmail(email) .orElseThrow(()
-> new RuntimeException("User not found")); } }
```

6 Controller Layer - API Endpoints

What is a Controller?

Controller = API Endpoints. Controllers receive HTTP requests from frontend, call services, and return responses. They are the "face" of your API.

Path: src/main/java/com/myapp/sponsorshipapp/controller/

Annotations:
• @RestController - Makes this an API controller
• @RequestMapping - Base URL path
• @GetMapping, @PostMapping, @PutMapping, @DeleteMapping - HTTP methods
• @RequestBody - Parse request body to DTO

Controller Classes

Controller	Base Path	Responsibility	Methods
AuthController	/api/auth	Login, signup, password	register, login, currentUser, changePassword
CampaignController	/api/campaigns	Campaign CRUD	create, update, delete, get, list
SponsorshipController	/api/influencer	Apply, work submission	apply, submitWork, checkApplications
PaymentController	/api/brand	Payment processing	processPayment, getPaymentStatus
RatingController	/api/campaigns/rate	Submit ratings	submitRating, getRatings

NotificationController	/api/notifications	Notifications	getNotifications, markAsRead
AdminController	/api/admin	Admin operations	getStats, allUsers, suspendUser

Example: AuthController - Complete

AuthController.java - API Endpoints

```
@RestController @RequestMapping("/api/auth") @CrossOrigin(origins = "http://localhost:4200") // Allow frontend public class AuthController { @Autowired private AuthService authService; // ===== POST /api/auth/register ===== @PostMapping("/register") public ResponseEntity<?> register(@Valid @RequestBody RegisterRequest request) { // @Valid validates RegisterRequest data // @RequestBody converts JSON to RegisterRequest object try { AuthResponse response = authService.register(request); return ResponseEntity.ok(response); // Returns 200 OK with token } catch (Exception e) { return ResponseEntity.badRequest() .body(new ApiResponse(false, e.getMessage())); // Returns 400 Bad Request with error message } } // ===== POST /api/auth/login ===== @PostMapping("/login") public ResponseEntity<?> login(@Valid @RequestBody LoginRequest request) { try { AuthResponse response = authService.login(request); return ResponseEntity.ok(response); // Returns 200 OK with token } catch (Exception e) { return ResponseEntity.badRequest() .body(new ApiResponse(false, "Invalid email or password")); // Returns 400 Bad Request } } // ===== GET /api/auth/me ===== // Gets current logged-in user info @GetMapping("/me") public ResponseEntity<?> getCurrentUser() { // JwtAuthenticationFilter already validated JWT // User is definitely authenticated try { User user = authService.getCurrentUser(); return ResponseEntity.ok(user); } catch (Exception e) { return ResponseEntity.badRequest() .body(new ApiResponse(false, e.getMessage())); } } // ===== POST /api/auth/change-password ===== @PostMapping("/change-password") public ResponseEntity<?> changePassword( @Valid @RequestBody ChangePasswordRequest request ) { try { authService.changePassword(request); return ResponseEntity.ok(new ApiResponse(true, "Password changed")); } catch (Exception e) { return ResponseEntity.badRequest() .body(new ApiResponse(false, e.getMessage())); } } }
```

Example: CampaignController - REST CRUD Operations

CampaignController.java - CRUD Pattern

```
@RestController @RequestMapping("/api/campaigns") public class
CampaignController { @Autowired private CampaignService
campaignService; // ===== CREATE: POST /api/campaigns =====
@PostMapping public ResponseEntity<?> createCampaign( @Valid
@RequestBody CampaignRequest request ) { Campaign campaign =
campaignService.createCampaign(request); return ResponseEntity.ok(
new ApiResponse(true, "Campaign created", campaign) ); } // =====
READ: GET /api/campaigns/{id} ===== @GetMapping("/{id}") public
ResponseEntity<Campaign> getCampaign(@PathVariable Long id) {
Campaign campaign = campaignService.getCampaignById(id); return
ResponseEntity.ok(campaign); } // ===== READ: GET /api/campaigns
===== @GetMapping public ResponseEntity<List<Campaign>>
getAllCampaigns() { return
ResponseEntity.ok(campaignService.getAllCampaigns()); } // =====
UPDATE: PUT /api/campaigns/{id} ===== @PutMapping("/{id}") public
ResponseEntity<?> updateCampaign( @PathVariable Long id, @Valid
@RequestBody CampaignRequest request ) { Campaign campaign =
campaignService.updateCampaign(id, request); return
ResponseEntity.ok( new ApiResponse(true, "Campaign updated",
campaign) ); } // ===== DELETE: DELETE /api/campaigns/{id} =====
@DeleteMapping("/{id}") public ResponseEntity<?>
deleteCampaign(@PathVariable Long id) {
campaignService.deleteCampaign(id); return ResponseEntity.ok( new
ApiResponse(true, "Campaign deleted") ); } } // REST Pattern: //
POST /api/campaigns - Create // GET /api/campaigns - List all //
GET /api/campaigns/{id} - Get one // PUT /api/campaigns/{id} -
Update // DELETE /api/campaigns/{id} - Delete
```

7 Security Package - Authentication

What is the Security Package?

The security package handles JWT token creation, validation, and user authentication. It works with Spring Security to protect your application.

Path: src/main/java/com/myapp/sponsorshipapp/security/

Contains 3 main classes:

• JwtTokenProvider - Creates & validates JWT

• JwtAuthenticationFilter - Runs on every request

• CustomUserDetailsService - Loads user from database

Security Classes

Class	Purpose	Key Methods
JwtTokenProvider.java	JWT token operations	generateToken(), validateToken(), getUsernameFromToken()
JwtAuthenticationFilter.java	Validate JWT on each request	doFilterInternal()
CustomUserDetailsService.java	Load user for authentication	loadUserByUsername()

Example: JwtTokenProvider - Token Creation

JwtTokenProvider.java - Token Management

```
@Component public class JwtTokenProvider {
@Value("${app.jwt.secret}") private String jwtSecret; //
From application.properties @Value("${app.jwt.expiration}")
private long jwtExpiration; // 86400000 = 24 hours // =====
GENERATE TOKEN ===== public String
generateToken(Authentication authentication) { UserDetails
userDetails = (UserDetails) authentication.getPrincipal();
Date now = new Date(); Date expiryDate = new
Date(now.getTime() + jwtExpiration); return Jwts.builder()
.subject(userDetails.getUsername()) // Email .issuedAt(now)
// When created .expiration(expiryDate) // When expires (24h
later) .signWith(getSigningKey()) // Sign with secret
.compact(); // Compress to string } // ===== VALIDATE TOKEN
===== public boolean validateToken(String token) { try {
Jwts.parser() .verifyWith(getSigningKey()) .build()
.parseSignedClaims(token); return true; // Valid! } catch
(JwtException | IllegalArgumentException e) { return false;
// Invalid! } } // ===== GET USERNAME FROM TOKEN =====
public String getUsernameFromToken(String token) { Claims
claims = Jwts.parser() .verifyWith(getSigningKey()) .build()
.parseSignedClaims(token) .getPayload(); return
claims.getSubject(); // Returns email } }
```

8 Exception Handling - Error Management

What is GlobalExceptionHandler?

Instead of handling errors in every method, Spring provides **@RestControllerAdvice** to handle all errors in one place globally.

GlobalExceptionHandler.java - Centralized Error Handling

```
@RestControllerAdvice public class GlobalExceptionHandler {
// ===== HANDLE RUNTIME EXCEPTIONS =====
@ExceptionHandler(RuntimeException.class) public
ResponseEntity<ApiResponse>
handleRuntimeException(RuntimeException ex) { return
ResponseEntity.badRequest() .body(new ApiResponse(false,
ex.getMessage())); // Any RuntimeException → 400 Bad Request
response } // ===== HANDLE BAD CREDENTIALS =====
@ExceptionHandler(BadCredentialsException.class) public
ResponseEntity<ApiResponse> handleBadCredentialsException(
BadCredentialsException ex ) { return
ResponseEntity.status(HttpStatus.UNAUTHORIZED) .body(new
ApiResponse(false, "Invalid email or password")); // Wrong
email/password → 401 Unauthorized } // ===== HANDLE
VALIDATION ERRORS =====
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Map<String, String>>
handleValidationExceptions( MethodArgumentNotValidException
ex ) { Map<String, String> errors = new HashMap<>();
ex.getBindingResult().getAllErrors().forEach((error) -> {
String fieldName = ((FieldError) error).getField(); String
errorMessage = error.getDefaultMessage();
errors.put(fieldName, errorMessage); }); return
ResponseEntity.badRequest().body(errors); // Validation
errors → 400 with field-level errors // Example: {"email":
"Invalid email format", "name": "Name required"} } } //
Benefits: // ✓ Consistent error responses // ✓ No need for
try-catch in every method // ✓ Frontend knows exactly what
to expect
```

9️⃣ Complete Data Flow with Examples

Example Flow: User Login

```
USER (Frontend)
    |
    | 1. Enters email & password
    | Clicks LOGIN button
    |
    | 2. Frontend sends HTTP POST
    | POST /api/auth/login
    | Content-Type: application/json
    | Body: {"email":"john@gmail.com","password":"pass123"}
    |
    ↓
AuthController.login()
    |
    | 3. @RequestBody → Converts JSON to LoginRequest DTO
    | LoginRequest validation (@NotBlank, @Email)
    |
    | 4. Calls authService.login(request)
    |
    ↓
AuthService.login()
    |
    | 5. Normalize email: "john@gmail.com"
    |
    | 6. AuthenticationManager.authenticate()
    |    ├─ Calls CustomUserDetailsService.loadUserByUsername("john@gmail.com")
    |    ├─ Repository searches database for user
    |    └─ Compares password using BCrypt
    |
    | 7. If password matches:
    |    ├─ Set SecurityContext
    |    ├─ Call tokenProvider.generateToken()
    |    └─ JWT token created!
    |
    | 8. Find user from database
    |    └─ userRepository.findByEmail("john@gmail.com")
    |
    | 9. Return AuthResponse with token
    |
```

```
    ↓
AuthController returns response
    |
    | 10. Convert AuthResponse to JSON
    | HTTP 200 OK
    | Body: {
    |   "token": "eyJhbGciOiJ...",
    |   "id": 1,
    |   "name": "John",
    |   "email": "john@gmail.com",
    |   "role": "INFLUENCER"
    | }
    |
    ↓
Frontend receives response
    |
    | 11. Save token: localStorage.setItem('jwt_token', token)
    | 12. Redirect to dashboard
    |
    ✓ USER LOGGED IN!
```

Example Flow: Create Campaign

```
BRAND USER (Frontend)
    |
    | 1. Fills campaign form:
    |    - Name: "Summer Fashion"
    |    - Platform: "Instagram"
    |    - Budget: 5000
    | Clicks CREATE button
    |
    | 2. Frontend sends HTTP POST
    | POST /api/campaigns
    | Authorization: Bearer eyJhbGciOiJ...  (JWT token)
    | Content-Type: application/json
    | Body: {
    |   "name": "Summer Fashion",
    |   "platform": "Instagram",
    |   "budget": 5000,
    |   ...
    | }
    |
    ↓
SecurityFilterChain (Automatically)
    |
    | 3. JwtAuthenticationFilter.doFilterInternal()
    |    ├─ Extract token from Authorization header
```

```
    |    ├─ Validate token signature
    |    ├─ Check if expired
    |    ├─ Extract email "brand@gmail.com"
    |    ├─ Load user from database
    |    └─ Set in SecurityContext
    |
    | 4. Check Authorization
    |    └─ Is user a BRAND? YES! ✓
    |
    ↓
CampaignController.createCampaign()
    |
    | 5. @RequestBody → Converts JSON to CampaignRequest DTO
    | CampaignRequest validation (@NotBlank, @Positive)
    |
    | 6. Calls campaignService.createCampaign(request)
    |
    ↓
CampaignService.createCampaign()
    |
    | 7. Get current brand user from SecurityContext
    |    └─ authService.getCurrentUser()
    |
    | 8. Create Campaign object
    |    ├─ Set name, platform, budget from request
    |    ├─ Set brand = current user
    |    └─ Set status = ACTIVE
    |
    | 9. Save to database
    |    └─ campaignRepository.save(campaign)
    |
    | 10. Create notification for brand
    |     └─ "Your campaign created successfully!"
    |
    | 11. Return Campaign object
    |
    ↓
CampaignController returns response
    |
    | 12. Convert Campaign to JSON
    | HTTP 200 OK
    | Body: {
    |   "success": true,
    |   "message": "Campaign created",
    |   "data": {
    |     "id": 15,
    |     "name": "Summer Fashion",
    |     "platform": "Instagram",
    |     "budget": 5000,
    |     "status": "ACTIVE",
    |     ...
    |   }
```

```
    | }
    |
    ↓
Frontend receives response
    |
    | 13. Show success message
    | 14. Redirect to campaigns list
    |
    ✓ CAMPAIGN CREATED!
```

🔟 API Endpoints Summary

Authentication Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/auth/register	Create account	No
POST	/api/auth/login	Login	No
GET	/api/auth/me	Get current user	Yes
POST	/api/auth/change-password	Change password	Yes

Campaign Endpoints

Method	Endpoint	Description	Role Required
POST	/api/campaigns	Create campaign	BRAND
GET	/api/campaigns	Get all campaigns	None
GET	/api/campaigns/{id}	Get campaign details	None
PUT	/api/campaigns/{id}	Update campaign	BRAND
DELETE	/api/campaigns/{id}	Delete campaign	BRAND
GET	/api/campaigns/active	Get active campaigns	None

Influencer Endpoints

Method	Endpoint	Description
POST	/api/influencer/apply	Apply to campaign
POST	/api/influencer/submit-work	Submit work proof

GET	/api/influencer/my-applications	View my applications

📚 Complete Architecture Summary

Package Responsibilities

🔹 Entity

Maps to database

6 tables + 4 enums

Represents domain objects

🔹 DTO

API communication

11 DTO classes

Simple data objects

🔹 Repository

Database queries

6 repositories

Spring Data JPA

🔹 Service

Business logic

7 services

All decisions here

🔹 Controller

API endpoints

7 controllers

Handle requests

🔹 Security

Authentication

3 classes

JWT tokens

Data Journey

```
Frontend (Angular)
    ↓ HTTP Request with JWT
Controller (Receives request)
    ↓ Validates DTO
Service (Business logic)
    ↓ Repository queries
Database (MySQL)
    ↓ Returns data
Repository (Wraps in Entity)
    ↓ Converts to DTO
Controller (Wraps in ApiResponse)
```

```
    ↓ HTTP Response
Frontend (Receives & displays)
```

✅ You Now Understand Backend Architecture!

You know:

✓ What each package does

✓ What each class is responsible for

✓ How they connect together

✓ How data flows through the system

✓ Complete API structure

✓ All 7 controllers and 7 services

✓ How validation and error handling works

✓ How authentication is implemented